

# Module Guide for Software Engineering

Team #16, The Chill Guys

Hamzah Rawasia

Sarim Zia

Aidan Froggatt

Swesan Pathmanathan

Burhanuddin Kharodawala

April 5, 2026

# 1 Revision History

| Date          | Developer           | Notes                            |
|---------------|---------------------|----------------------------------|
| November 1st  | Aidan Froggatt      | Completed Section 11             |
| November 12th | Hamzah Rawasia      | Completed Sections 6 and 8       |
| November 12th | Swesan Pathmanathan | Completed Sections 5 and 7       |
| November 13th | Burhan Kharodawala  | Completed Sections 3, 4, and 10  |
| November 13th | Sarim Zia           | Completed Sections 9, 10, and 12 |

## 2 Reference Material

This section records information for easy reference.

### 2.1 Abbreviations and Acronyms

| symbol               | description                         |
|----------------------|-------------------------------------|
| AC                   | Anticipated Change                  |
| DAG                  | Directed Acyclic Graph              |
| M                    | Module                              |
| MG                   | Module Guide                        |
| OS                   | Operating System                    |
| R                    | Requirement                         |
| SC                   | Scientific Computing                |
| SRS                  | Software Requirements Specification |
| Software Engineering | Explanation of program name         |
| UC                   | Unlikely Change                     |

# Contents

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| <b>1</b>  | <b>Revision History</b>                           | <b>i</b>  |
| <b>2</b>  | <b>Reference Material</b>                         | <b>ii</b> |
| 2.1       | Abbreviations and Acronyms . . . . .              | ii        |
| <b>3</b>  | <b>Introduction</b>                               | <b>1</b>  |
| <b>4</b>  | <b>Anticipated and Unlikely Changes</b>           | <b>2</b>  |
| 4.1       | Anticipated Changes . . . . .                     | 2         |
| 4.2       | Unlikely Changes . . . . .                        | 2         |
| <b>5</b>  | <b>Module Hierarchy</b>                           | <b>3</b>  |
| <b>6</b>  | <b>Connection Between Requirements and Design</b> | <b>4</b>  |
| <b>7</b>  | <b>Traceability Matrix</b>                        | <b>5</b>  |
| <b>8</b>  | <b>Use Hierarchy Between Modules</b>              | <b>7</b>  |
| <b>9</b>  | <b>User Interfaces</b>                            | <b>7</b>  |
| <b>10</b> | <b>Design of Communication Protocols</b>          | <b>25</b> |
| 10.1      | 11.1 Overall Architecture . . . . .               | 25        |
| 10.2      | 11.2 Communication Flow . . . . .                 | 26        |
| 10.3      | 11.3 API Endpoints and Message Types . . . . .    | 26        |
| 10.4      | 11.4 Security and Reliability . . . . .           | 27        |
| 10.5      | 11.5 Design Considerations . . . . .              | 27        |
| <b>11</b> | <b>Timeline</b>                                   | <b>28</b> |

## List of Tables

|   |                                                                   |    |
|---|-------------------------------------------------------------------|----|
| 1 | Module Hierarchy . . . . .                                        | 4  |
| 2 | Trace Between Requirements and Modules . . . . .                  | 6  |
| 3 | Trace Between Anticipated Changes and Modules . . . . .           | 7  |
| 4 | Phase 1 . . . . .                                                 | 28 |
| 5 | Phase 2: Front End and User Roles . . . . .                       | 28 |
| 6 | Phase 3: Core Ride-Share Logic and Trip Flow . . . . .            | 29 |
| 7 | Phase 4: Safety, Database Integration, and Unit Testing . . . . . | 29 |
| 8 | Phase 5: Integration Testing and Finalization . . . . .           | 30 |

## List of Figures

|    |                                                    |    |
|----|----------------------------------------------------|----|
| 1  | Use hierarchy among modules . . . . .              | 7  |
| 2  | Main Login Screen . . . . .                        | 8  |
| 3  | Register Screen . . . . .                          | 9  |
| 4  | Verification Screen . . . . .                      | 10 |
| 5  | Upload Schedule Screen . . . . .                   | 11 |
| 6  | Select User Type Screen . . . . .                  | 12 |
| 7  | User Prompted to Add Profile Information . . . . . | 13 |
| 8  | Home Screen . . . . .                              | 14 |
| 9  | Switch to Driver Screen . . . . .                  | 15 |
| 10 | Switch to Rider Screen . . . . .                   | 16 |
| 11 | Driver Connection Screen . . . . .                 | 17 |
| 12 | Rider Connection Screen . . . . .                  | 18 |
| 13 | Matched Rider List Screen . . . . .                | 19 |
| 14 | Matched Driver List Screen . . . . .               | 20 |
| 15 | No Matches Found Screen . . . . .                  | 21 |
| 16 | No Matches Found Screen . . . . .                  | 22 |
| 17 | Driver Navigation Screen . . . . .                 | 23 |
| 18 | Rider Wait Screen . . . . .                        | 24 |
| 19 | Cost Estimation Screen . . . . .                   | 25 |

### 3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (Parnas et al., 1984). We advocate a decomposition based on the principle of information hiding (Parnas, 1972). This principle supports design for change, because the “secrets” that each module hides represent likely future changes. Design for change is valuable in SC, where modifications are frequent, especially during initial development as the solution space is explored.

Our design follows the rules laid out by Parnas et al. (1984), as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module’s data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed (Parnas et al., 1984). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New project members: This document can be a guide for a new project member to easily understand the overall structure and quickly find the relevant modules they are searching for.
- Maintainers: The hierarchical structure of the module guide improves the maintainers’ understanding when they need to make changes to the system. It is important for a maintainer to update the relevant sections of the document after changes have been made.
- Designers: Once the module guide has been written, it can be used to check for consistency, feasibility, and flexibility. Designers can verify the system in various ways, such as consistency among modules, feasibility of the decomposition, and flexibility of the design.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the likely changes. Section 6 specifies the connections between the software requirements and the modules. Section ?? gives a detailed description of the modules. Section 7 includes two traceability matrices. One checks the completeness of the design against the requirements provided in the SRS. The other shows the relation between anticipated changes and the modules. Section 8 describes the use relation between modules.

## 4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

### 4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision. The approach adapted here is called design for change.

**AC1: Matching Algorithm:** The matching algorithm focuses on the goal of finding the best available drivers for the riders. This algorithm takes into account the user's location, timetable, time, and distance. For instance, it matches a rider with a driver that has the same timetable and lives in the same area. The current algorithm is based on these parameters, but it will likely change in the future to consider additional variables to increase efficiency and accuracy. Moreover, subtle changes to the heuristics may be expected for further optimization of complexity and performance.

**AC2: User Interface:** As the application proceeds through its initial phase of development and use, usability testing is expected to ensure user satisfaction. As users provide feedback, changes will be made to the user interface in future phases. Moreover, changes may be required to meet evolving accessibility standards. The layout and navigation workflows are the primary components of user interaction that are expected to evolve based on user feedback.

**AC3: Data Model and Schema:** As the application updates with new features, the underlying data structures may change. New parameters would be considered for the matching algorithm, and new entities may be added based on future user needs. Therefore, changes to the persistent storage schema and data definitions are anticipated.

**AC4: Payment Processing:** Additional payment processors and pricing formulas may be integrated to determine price estimates for drivers. An addition of parameters to the cost estimation logic would cause the underlying algorithms to change.

**AC5: Operational Configuration:** Changes could be made to the build pipelines and deployment configurations to improve the reliability, stability, and automation of the system.

### 4.2 Unlikely Changes

The module design should be as general as possible. However, a general system is more complex. Sometimes this complexity is not necessary. Fixing some design decisions at the

system architecture stage can simplify the software design. If these decisions should later need to be changed, then many parts of the design will potentially need to be modified. Hence, it is not intended that these decisions will be changed.

**UC1: Target Platform:** Hitchly is an application developed specifically for mobile devices. Porting the system to a desktop-native environment is an unlikely change as it would require significant modifications to the user interaction design and location-based services.

**UC2: Language Support:** The application is primarily based on the English language. Architecting the interface to support multi-language localization would require a major structural design decision.

**UC3: Implementation Technologies:** The core of this application is developed using a specific set of cross-platform frameworks and typed languages. Changing the fundamental development stack or database engine is an unlikely change as it would require a complete rewrite of the application’s codebase.

**UC4: Verification Requirement:** This application is strictly restricted for users affiliated with the specific institution (McMaster University), as the core domain concept relies on this trusted community. Expanding to the general public is not subject to change. Verification of users will consistently rely on institutional email validation.

## 5 Module Hierarchy

This section provides an overview of the modular design for Hitchly. The hierarchy follows the principle of information hiding, where each module encapsulates a design decision that may change independently. Only the leaf modules shown in Table 1 are implemented.

**5.1 Hardware-Hiding Modules** There are no hardware-hiding modules in this design. Hitchly relies on high-level framework APIs (Expo, React Native), so no direct hardware abstraction is required.

**5.2 Behaviour-Hiding Modules** These modules implement the user-visible behaviour of the application. They hide workflow logic, API interactions, and data handling that support Hitchly’s core features.

- M1: Authentication & Verification Module
- M2: User Profile Module
- M3: Route & Trip Module
- M6: Notification Module
- M7: Rating & Feedback Module



- M8: Safety & Reporting Module
- M9: Payment & Cost Estimation Module
- M10: Admin & Moderation Module

**5.3 Software-Decision Modules** These modules encapsulate internal logic, algorithms, and implementation decisions not visible to users.

- M4: Matching Module
- M5: Scheduling Module
- M11: Pricing Module
- M12: Database Module

| Level 1                  | Level 2                                  |
|--------------------------|------------------------------------------|
| Hardware-Hiding Module   | None                                     |
| Behaviour-Hiding Module  | M1: Authentication & Verification Module |
|                          | M2: User Profile Module                  |
|                          | M3: Route & Trip Module                  |
|                          | M6: Notification Module                  |
|                          | M7: Rating & Feedback Module             |
|                          | M8: Safety & Reporting Module            |
|                          | M9: Payment & Cost Estimation Module     |
| Software-Decision Module | M10: Admin & Moderation Module           |
|                          | M4: Matching Module                      |
|                          | M5: Scheduling Module                    |
|                          | M11: Pricing Module                      |
|                          | M12: Database Module                     |

Table 1: Module Hierarchy

## 6 Connection Between Requirements and Design

The design of the system is intended to satisfy the requirements developed in the SRS. In this stage, the system is decomposed into modules. The connection between requirements and modules is listed in Table 2.

## 7 Traceability Matrix

This section shows two traceability matrices: between the modules and the requirements and between the modules and the anticipated changes.

| <b>Req.</b> | <b>Modules</b>  |
|-------------|-----------------|
| FR211       | M4, M3, M5, M10 |
| FR212       | M4, M5, M10     |
| FR213       | M4, M2          |
| FR214       | M4, M2, M5      |
| NFR211      | M4, M10         |
| NFR212      | M4, M1, M10     |
| FR221       | M5, M3, M10     |
| FR222       | M5, M6          |
| FR223       | M5, M6, M9      |
| FR224       | M6, M5          |
| NFR221      | M10, M5         |
| NFR222      | M6, M5, M10     |
| FR231       | M1, M2          |
| FR232       | M1, M10         |
| FR233       | M8, M3, M6      |
| FR234       | M8, M2, M12     |
| NFR231      | M1, M10         |
| NFR232      | M1, M10, M12    |
| FR241       | M11, M5, M3     |
| FR242       | M9              |
| FR243       | M9              |
| FR244       | M10, M9         |
| NFR241      | M9              |
| NFR242      | M9              |
| FR251       | M2, M1          |
| FR252       | M10, M2         |
| FR253       | M10, M2, M3     |
| FR254       | M2, M4          |
| NFR251      | M2, M1          |
| NFR252      | M2, M10         |
| FR261       | M7, M2          |
| FR262       | M8, M12         |
| FR263       | M7, M5, M4      |
| FR264       | M7, M2          |
| NFR261      | M7, M12         |

Table 2: Trace Between Requirements and Modules

| AC  | Modules                |
|-----|------------------------|
| AC1 | M4, M2, M5             |
| AC2 | M2, M3, M4             |
| AC3 | M10                    |
| AC4 | M9, M11                |
| AC5 | N/A (External Process) |

Table 3: Trace Between Anticipated Changes and Modules

## 8 Use Hierarchy Between Modules

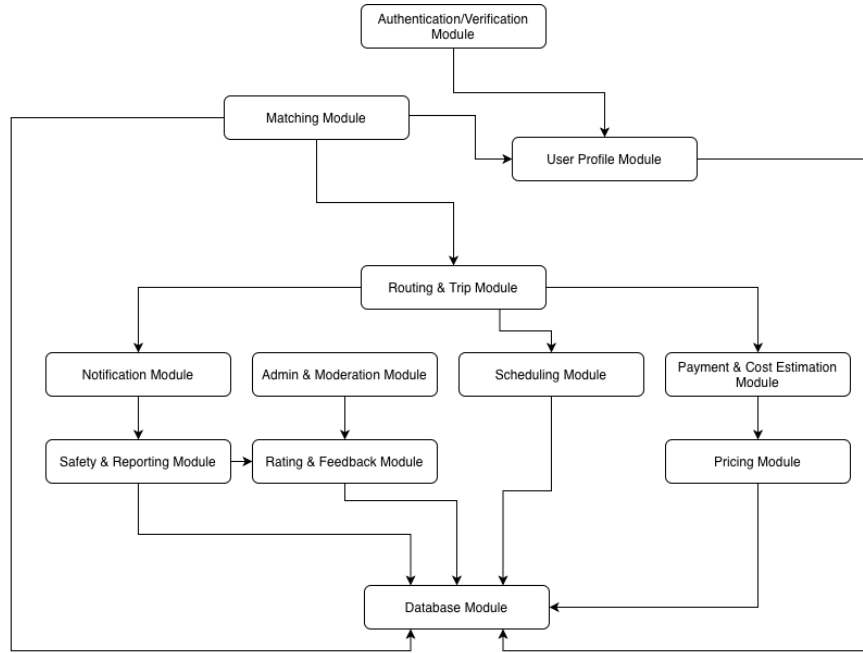


Figure 1: Use hierarchy among modules

## 9 User Interfaces

This section provides an overview of the important screens of the application.



Figure 2: Main Login Screen

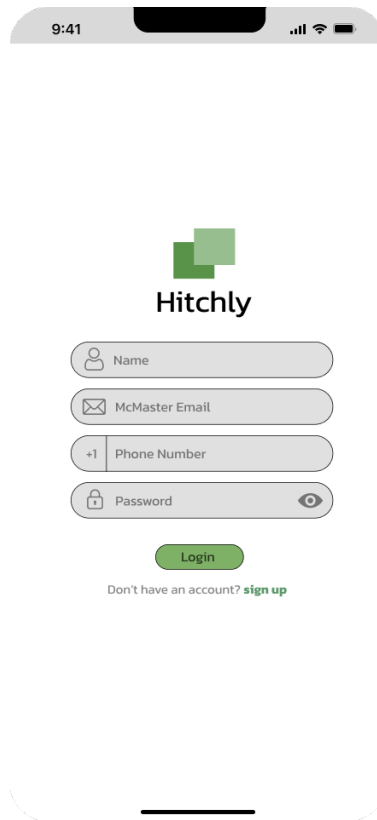


Figure 3: Register Screen

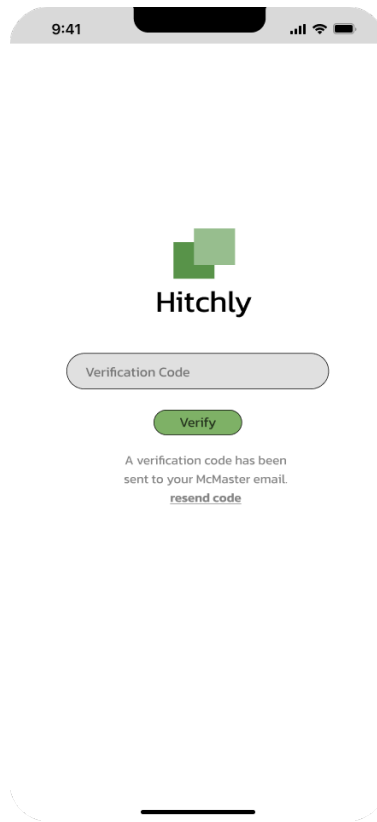


Figure 4: Verification Screen

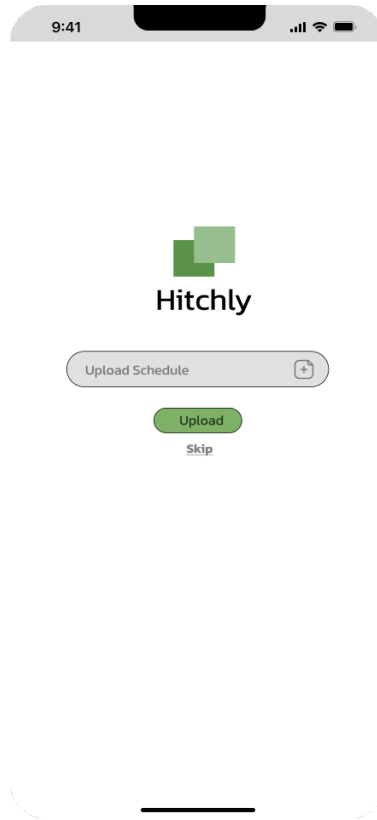


Figure 5: Upload Schedule Screen



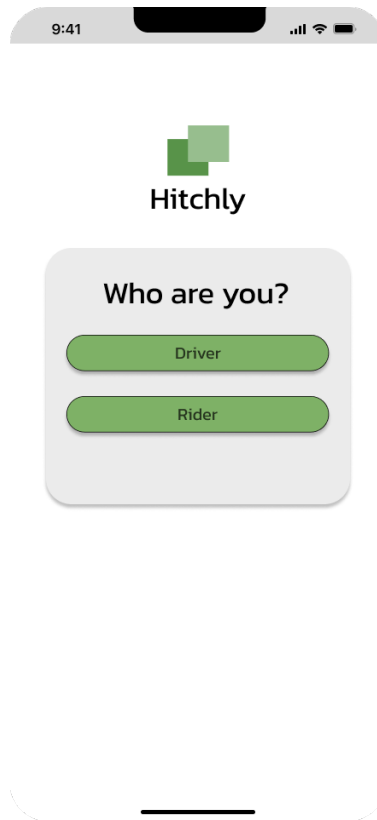


Figure 6: Select User Type Screen

The image shows a mobile application interface for a 'User Profile'. At the top, a status bar displays the time '9:41' and signal icons. Below this, a grey header bar contains the text 'User Profile'. The main content area is a light grey card. At the top of the card is a circular profile picture placeholder with a person icon. Below the placeholder is the text 'Change/Add Profile Picture'. The card contains six rounded rectangular input fields, each with a label: 'First Name', 'Last Name', 'Email', 'Phone Number', 'Age', and 'Gender'. At the bottom right of the card is a green rounded rectangular button labeled 'Next'. The entire screen is framed by a white border with a thin black line at the bottom.

Figure 7: User Prompted to Add Profile Information

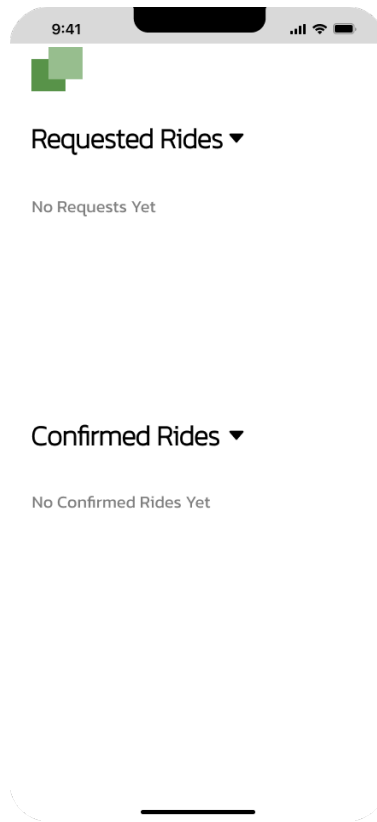


Figure 8: Home Screen

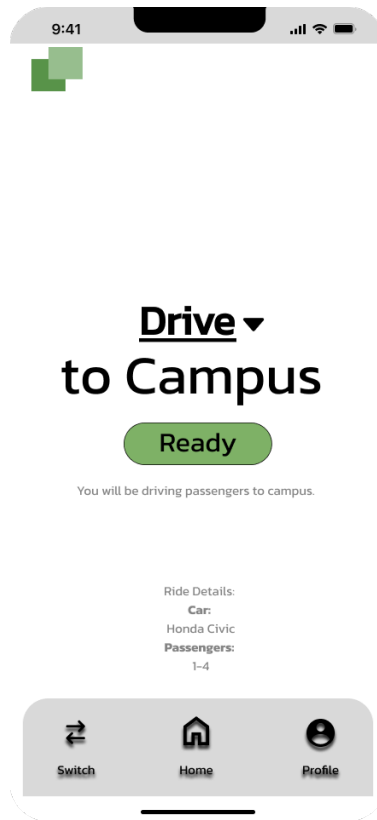


Figure 9: Switch to Driver Screen

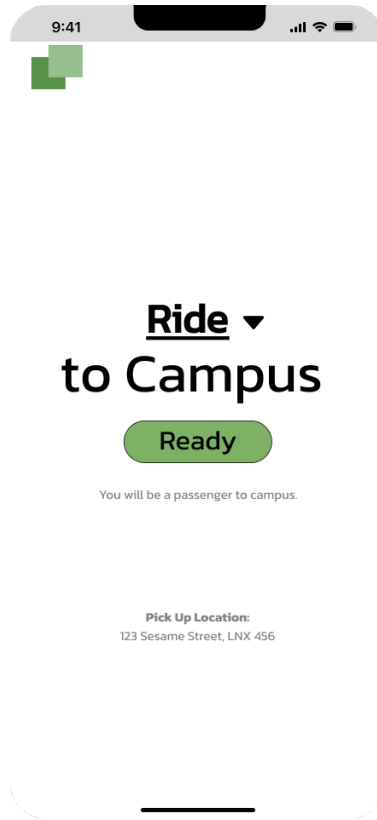


Figure 10: Switch to Rider Screen

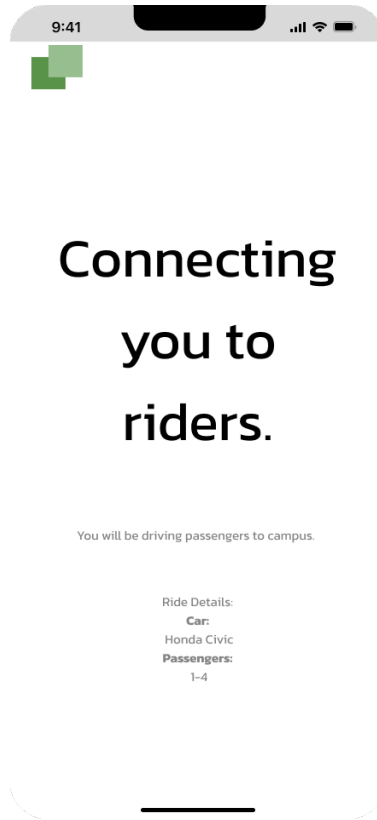


Figure 11: Driver Connection Screen

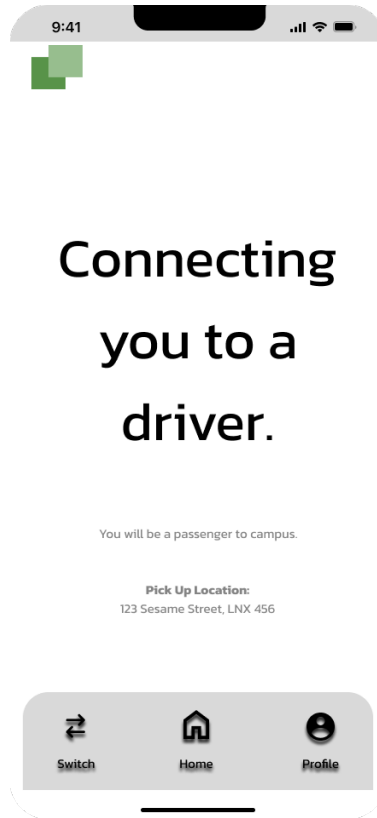


Figure 12: Rider Connection Screen

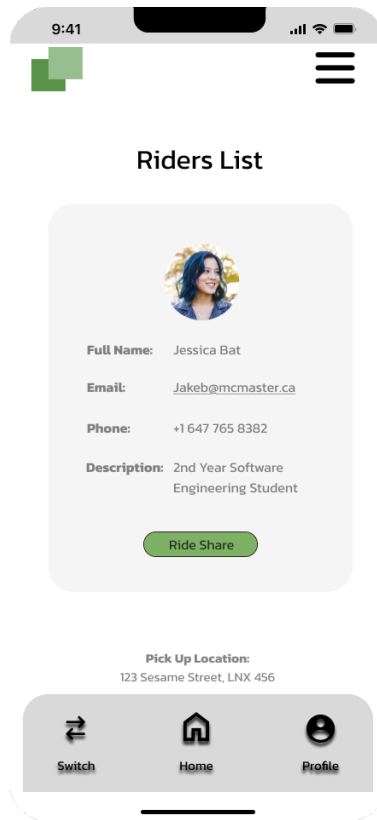


Figure 13: Matched Rider List Screen



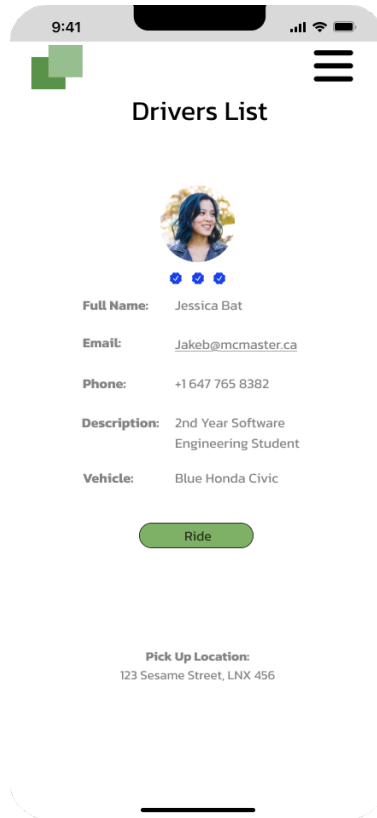


Figure 14: Matched Driver List Screen

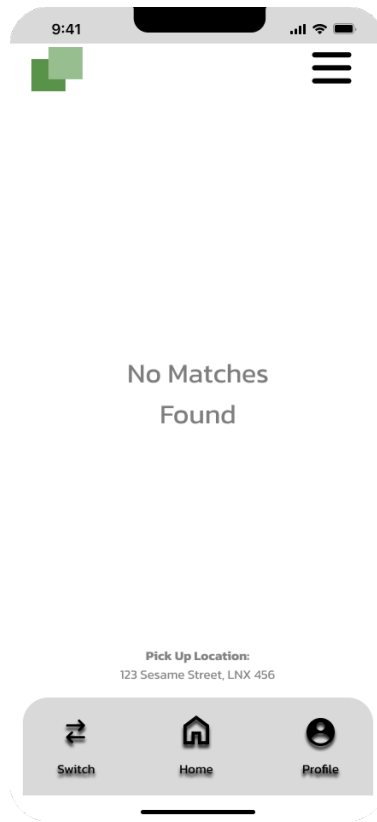


Figure 15: No Matches Found Screen

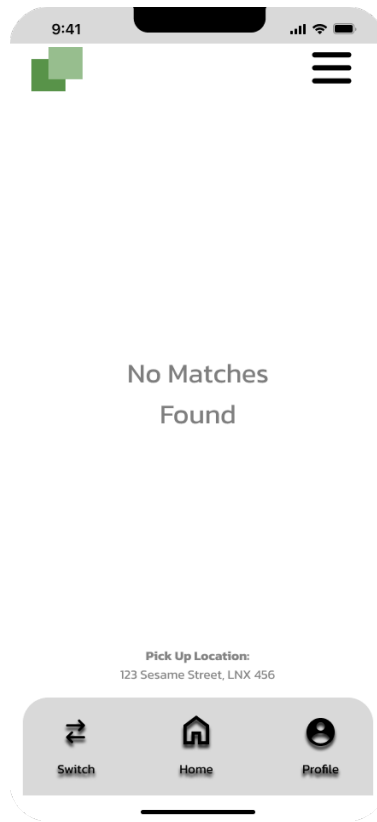


Figure 16: No Matches Found Screen

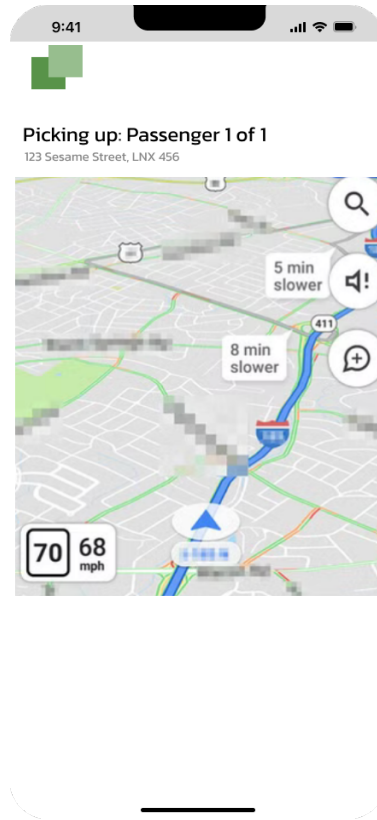


Figure 17: Driver Navigation Screen



Figure 18: Rider Wait Screen

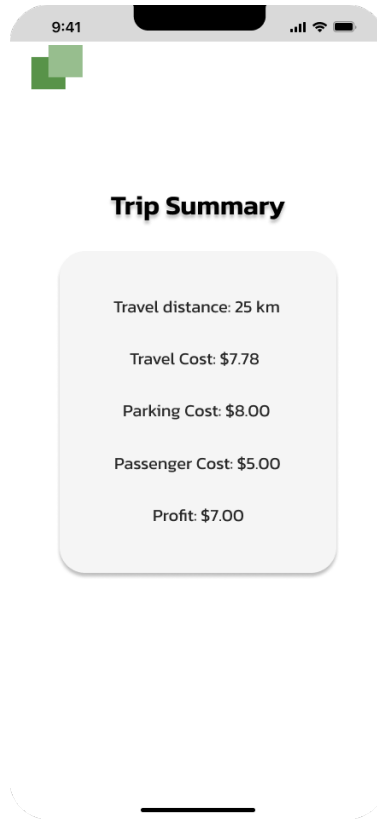


Figure 19: Cost Estimation Screen

## 10 Design of Communication Protocols

This section describes the design of the communication protocols implemented in Hitchly. Communication occurs primarily between the client-side mobile application and the backend server infrastructure. The system also integrates third-party interfaces for geolocation and external notification services.

### 10.1 11.1 Overall Architecture

All communication between the mobile client and the backend follows a request-response model over secure transport channels, ensuring confidentiality and integrity. Data is serialized into a structured format and transferred through defined API endpoints. The architecture adopts a modular service-based design to support scalability and maintainability:

- **Frontend (Client):** Mobile application interface that sends authenticated requests to backend endpoints via secure transport protocols.
- **Backend (Server):** Server-side application exposing routes through an API layer. It validates and processes incoming requests, interacts with the persistence layer, and

returns structured responses.

- **External Services:** Interfaces for mapping and geolocation data, push notification services for alerts, and identity verification providers (e.g., institutional SSO for email validation).

## 10.2 11.2 Communication Flow

A typical data exchange involves the following sequence:

1. The user initiates an action in the mobile application (e.g., requests a ride, updates schedule, or rates a driver).
2. The frontend constructs a remote procedure call encapsulating the action data and attaches the user's authentication credentials in the header.
3. The server authenticates the request, validates input schemas against defined contracts, and executes the corresponding service logic.
4. The backend queries or updates the persistence layer and returns a success or error payload to the client.
5. The client updates its local state and user interface based on the received data.

## 10.3 11.3 API Endpoints and Message Types

Hitchly's backend exposes the following logical API groups:

- **Auth API:** Handles user registration, login, institutional email verification, and session management. Exchanges encrypted credentials and session tokens.
- **User API:** Manages user profiles, schedules, and preferences.
- **Matching API:** Accepts commute parameters, executes the matching algorithm, and returns ranked ride offers.
- **Trip API:** Records completed trips, generates summaries, and calculates cost-sharing.
- **Rating API:** Submits and retrieves reputation scores and text reviews.
- **Notification API:** Interfaces with external push services to deliver trip confirmations, cancellations, and real-time updates.

All messages follow a standardized structure containing the operation status, payload data, and metadata:

```
{
  "status": "success",
  "data": { ... },
  "timestamp": "YYYY-MM-DDTHH:MM:SSZ"
}
```

## 10.4 11.4 Security and Reliability

Security and reliability are ensured through:

- **Transport Security:** Encryption in transit using TLS standards.
- **Authentication:** Bearer tokens or equivalent session credentials included in headers of all authorized requests.
- **Input Validation:** All incoming payloads are validated against strict schemas to prevent injection attacks or malformed data.
- **Error Handling:** Standardized error codes with human-readable messages returned to the client.
- **Rate Limiting:** Middleware limits excessive requests to prevent abuse and ensure service availability.
- **Retry Mechanism:** Client-side logic automatically retries transient network failures to improve resilience.

## 10.5 11.5 Design Considerations

The communication design of Hitchly emphasizes the following principles:

- **Consistency:** Shared type definitions across frontend and backend guarantee end-to-end type safety and contract adherence.
- **Extensibility:** Additional endpoints (e.g., payment interfaces or advanced analytics) can be introduced without modifying existing client modules.
- **Scalability:** The stateless architectural design supports horizontal scaling of the application server.
- **Low Latency:** Lightweight message serialization and asynchronous notification delivery minimize perceived user delay.

This design ensures secure, efficient, and maintainable communication between Hitchly's mobile application, backend, and external services, supporting its goals of safety, reliability, and sustainability for commuters.



## 11 Timeline

The following timeline details our implementation for Rev 0. Each task is broken down to the module level. Each task has roles and responsibilities defined along with estimated development duration.

Table 4: Phase 1

| Task                                        | Module(s) | Status | Responsible   |
|---------------------------------------------|-----------|--------|---------------|
| Creating Hitchly Wireframes for UI          | M2        | Done   | Burhan, Sarim |
| Make initial version of matching algorithm  | M4        | Done   | Hamzah, Sarim |
| Set up authentication and application bones | M1        | Done   | Aidan         |

Table 5: Phase 2: Front End and User Roles

| Task                                                                    | Module(s) | Status      | Responsible |
|-------------------------------------------------------------------------|-----------|-------------|-------------|
| Develop login, registration, and session flows                          | M1        | Weeks 11–12 | Aidan       |
| Build user profile screens and preferences interface                    | M2        | Weeks 11–12 | Swesan      |
| Create upload, settings, and account management pages                   | M2        | Weeks 11–12 | Burhan      |
| Build navigation, layouts, and key frontend components                  | M2        | Weeks 12–13 | Hamzah      |
| Integrate basic user-to-database connections for profile initialization | M12       | Weeks 12–13 | Sarim       |

Table 6: Phase 3: Core Ride-Share Logic and Trip Flow

| Task                                                                | Module(s) | Status      | Responsible   |
|---------------------------------------------------------------------|-----------|-------------|---------------|
| Implement matching algorithm and rider-driver compatibility logic   | M4        | Weeks 13–14 | Sarim, Hamzah |
| Build Routing & Trip creation (origin, destination, seat count)     | M3        | Weeks 13–14 | Burhan        |
| Implement Scheduling module (recurring rides, time windows)         | M5        | Weeks 13–14 | Aidan         |
| Integrate Payment & Cost estimation (distance, time, pricing model) | M9        | Weeks 13–14 | Swesan        |
| Build Pricing module (fare calculations, dynamic pricing rules)     | M11       | Weeks 13–14 | Swesan        |
| Add Rating & Feedback system for both riders and drivers            | M7        | Weeks 13–14 | Sarim         |
| Implement Admin & Moderation tools (flagging, dispute handling)     | M10       | Weeks 13–14 | Burhan, Aidan |
| Build Notification module (push/in-app updates for trips)           | M6        | Weeks 13–14 | Hamzah        |

Table 7: Phase 4: Safety, Database Integration, and Unit Testing

| Task                                                             | Module(s) | Status      | Responsible  |
|------------------------------------------------------------------|-----------|-------------|--------------|
| Implement Safety & Reporting module (incident reports, flagging) | M8        | Weeks 14–15 | Burhan       |
| Complete full Database Module integration across all features    | M12       | Weeks 14–15 | Aidan, Sarim |
| Unit testing for Matching & Rating modules                       | M4, M7    | Weeks 14–15 | Sarim        |
| Unit testing for Auth & Scheduling modules                       | M1, M5    | Weeks 14–15 | Aidan        |
| Unit testing for Routing & Safety modules                        | M3, M8    | Weeks 14–15 | Burhan       |
| Unit testing for Payment & Pricing modules                       | M9, M11   | Weeks 14–15 | Swesan       |
| Unit testing for Notification module & Core UI Components        | M2, M6    | Weeks 14–15 | Hamzah       |

Table 8: Phase 5: Integration Testing and Finalization

| <b>Task</b>                                                   | <b>Module(s)</b> | <b>Status</b> | <b>Responsible</b> |
|---------------------------------------------------------------|------------------|---------------|--------------------|
| Module Testing & Bug Fixing for Safety                        | M8               | Weeks 15–16   | Burhan             |
| Integration testing: Database operations across all modules   | M12              | Weeks 15–16   | Aidan, Sarim       |
| Integration testing: Trip creation to Matching algorithm flow | M3, M4           | Weeks 15–16   | Sarim, Burhan      |
| Integration testing: Notifications & Scheduling triggers      | M5, M6           | Weeks 15–16   | Aidan, Hamzah      |
| Integration testing: Payment processing & Pricing UI feedback | M9, M11          | Weeks 15–16   | Swesan             |
| Perform regression testing and Finalize UI/UX                 | M2               | Weeks 16–17   | Hamzah, Swesan     |
| Prepare release build, deployment setup, and CI/CD pipelines  | System           | Weeks 18–19   | Aidan              |
| Final Design Documentation & Traceability review              | System           | Weeks 18–19   | Sarim, Burhan      |

## References

- David L. Parnas. On the criteria to be used in decomposing systems into modules. *Comm. ACM*, 15(2):1053–1058, December 1972.
- D.L. Parnas, P.C. Clement, and D. M. Weiss. The modular structure of complex systems. In *International Conference on Software Engineering*, pages 408–419, 1984.